

DBHub: 운영 DB 접근을 통제 가능한 업무 행위로 만들기

운영 DB 접근은 편의 기능이 아니라 권한, 승인, 감사, 제한, 복구 가능성을 갖춘 업무 행위다.

요약

DBHub는 운영 DB 접근을 SQL console이 아니라 통제 가능한 업무 행위로 다루기 위해 설계한 구조다. datasource 등록, SQL template, parameter schema, result schema, masking, 승인, audit, revision, canonical hash, 실행 trace를 하나의 흐름으로 묶었다.

핵심 판단은 명확했다. 운영 DB를 빠르게 조회하는 도구는 만들기 쉽지만, 장기적으로 필요한 것은 누가 어떤 목적으로 어떤 데이터를 조회·변경했고 어떤 결과가 나왔는지 설명할 수 있는 체계다.

문제 정의

운영 조직에서 DB 접근은 피할 수 없다. 장애 분석, 고객 문의, 데이터 보정, 배치 결과 확인, 외부 연동 점검을 위해 DB를 확인해야 한다. 그러나 일반 SQL tool을 운영 접근 경로로 사용하면 다음 문제가 발생한다.

- 권한과 목적이 SQL 실행 시점에 분리되지 않는다.
- 민감 데이터 masking이 일관되게 적용되지 않는다.
- 누가 어떤 SQL을 실행했는지 audit이 부족하다.
- 변경 SQL의 승인과 rollback 기준이 명확하지 않다.
- 동일한 운영 조회가 개인별 SQL로 흩어진다.
- AI가 SQL을 생성할 경우 통제 지점이 사라진다.

책임 경계

계층	책임
Datasource Registry	DB 연결 정보와 사용 범위를 관리한다.
Query Template	허용된 SQL과 parameter schema를 정의한다.
Policy	권한, stage, tenant, masking, row limit, timeout을 적용한다.

계층	책임
Approval	위험한 조회와 변경 작업을 승인 흐름에 올린다.
Audit	실행자, 목적, SQL hash, parameter, 결과 요약을 남긴다.
PlanVM	통제된 실행 runtime으로 DBHub 작업을 실행한다.

핵심 설계 결정

DB tool이 아니라 운영 데이터 접근 계층으로 보았다

DBHub는 자유 SQL 편집기를 목표로 하지 않았다. 자유 SQL은 표현 범위가 넓지만 운영 통제와 충돌한다. DBHub에서는 반복되는 운영 조회와 보정 작업을 template화하고, parameter schema와 result schema를 부여했다.

```
query template
-> parameter schema validation
-> permission check
-> masking / row limit / timeout
-> execution
-> audit / result summary
```

이 흐름을 통해 DB 접근을 개인의 기술 행위가 아니라 조직이 통제할 수 있는 업무 행위로 전환했다.

canonical hash와 revision을 사용했다

SQL template은 조금만 바뀌어도 결과와 위험도가 달라진다. 공백이나 주석 차이로 audit이 분산되면 추적이 어려워진다. DBHub는 SQL의 canonical form과 hash를 기준으로 revision을 관리하도록 설계했다.

이 기준은 승인과 audit에 중요하다. 승인된 SQL과 실제 실행된 SQL이 같은지 확인할 수 있어야 하며, 변경된 SQL은 별도 revision으로 기록되어야 한다.

조회와 변경을 다른 등급으로 다뤘다

조회 SQL과 변경 SQL은 위험도가 다르다. 조회도 개인정보와 민감 데이터 노출 위험이 있지만, 변경 SQL은 데이터 정합성과 업무 결과를 직접 바꾼다. DBHub에서는 조회와 변경을 같은 흐름으로 단순화하지 않았다.

- 조회: 권한, masking, row limit, timeout, 목적 기록을 적용한다.
- 변경: 사전 diff, 영향 범위, 승인, 실행 결과, rollback 기준을 요구한다.

- 대량 처리: batch 단위, dry-run, sample 검증, 실패 재처리를 분리한다.

Oracle과 PostgreSQL 차이를 숨기지 않았다

DBHub는 여러 DB를 동일한 UI로 다룰 수 있어야 하지만, DB별 차이를 무시하면 장애가 발생한다. Oracle과 PostgreSQL은 pagination, lock, transaction, sequence, data type, explain 결과가 다르다. 공통 표면을 제공하되 DB별 차이를 정책과 adapter에서 명시적으로 처리하는 방향이 필요했다.

버린 선택과 이유

DB 접속 정보를 나눠 주고 client tool 사용을 허용하는 방식은 운영 속도는 빠르지만 통제 기준이 약하다. 누가 어떤 SQL을 실행했는지, 결과를 어디까지 보았는지, 변경이 어떤 업무 데이터에 영향을 줬는지 설명하기 어렵다.

조회와 변경을 같은 등급으로 다루는 방식도 제외했다. 조회는 masking, row limit, export 제한이 중요하고, 변경은 승인, dry-run, 영향 row 수, rollback 가능성이 중요하다. 두 행위를 같은 버튼과 같은 권한으로 처리하면 위험하다.

LLM이 직접 SQL을 생성하고 실행하는 방식도 제한했다. 자연어가 편리하더라도 운영 DB는 실험 대상이 아니다. DBHub의 기준은 “질문에 빠르게 답하는 도구”가 아니라 “허용된 범위에서 추적 가능한 데이터 접근”이다.

운영자가 확인해야 하는 DB 접근 흔적

DBHub에서 중요한 것은 SQL 실행 성공 여부가 아니다. 운영자가 나중에 같은 결정을 검증할 수 있어야 한다.

흔적	설명
요청 목적	단순 조회, 장애 분석, 데이터 보정, 검증 query를 구분한다.
권한 근거	사용자, 역할, 앱, stage, datasource 접근 범위를 기록한다.
masking 결과	원본 데이터 노출과 화면 표시 결과를 분리한다.
영향 범위	update/delete 전 예상 row 수와 실제 row 수를 비교한다.
revision/hash	승인된 query와 실행된 query가 같은지 확인한다.

AI 시대의 DBHub

AI가 SQL을 생성할 수 있게 되면 DB 접근 통제는 더 중요해진다. LLM이 생성한 SQL을 사람이 복사해 운영 DB에서 실행하는 구조는 audit와 권한을 우회한다. DBHub는 AI가 제안한 SQL도 template, schema, permission, masking, approval, audit 흐름을 통과하도록 만드는 경계가 될 수 있다.

AI는 SQL 후보를 만들 수 있지만, 운영 DB 실행은 통제된 실행 계층을 통과해야 한다. 이 기준은 DBHub의 핵심 방향과 맞닿아 있다.

시스템 설계자의 그림

DBHub의 경계는 SQL 실행 버튼 앞에 있지 않다. 자연어 요청이나 운영자 요청이 SQL 후보로 바뀌는 순간부터 목적, 권한, datasource 범위, 민감도, 변경 위험, 결과 노출 범위가 함께 결정되어야 한다.

운영 검증 {#운영-검증 }

검증 기준은 “실행 가능한가”가 아니라 “운영자가 신뢰하고 감사할 수 있는가”였다. 자유도를 너무 높이면 DB tool이 되고, 너무 낮추면 실제 장애 대응에 사용되지 않는다. 따라서 반복 조회는 빠르게 실행하되, 민감하거나 위험한 작업은 승인과 제한을 걸었다.

주요 검증 항목은 다음과 같았다.

- 민감 컬럼 masking이 일관되게 적용되는가.
- row limit과 timeout이 우회되지 않는가.
- SQL 변경 시 revision과 hash가 남는가.
- 변경 작업은 승인과 결과 요약을 남기는가.
- 장애 모니터링 이벤트와 DB 조회 이력이 연결되는가.

역할과 책임

DBHub의 문제 정의, datasource registry, SQL template, parameter/result schema, canonical hash, revision, masking, approval, audit, PlanVM 실행 연동 방향을 설계했다.

설계 결론

운영 DB 접근은 편의성과 통제 사이에서 균형이 필요하다. DBHub는 SQL 실행기를 더 편하게 만드는 프로젝트가 아니라, 운영 DB 접근을 조직이 설명하고 감사할 수 있는 업무 행위로 바꾸는 구조다.